САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Выполнил:

Федоров Даниил

Группа

K3440

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

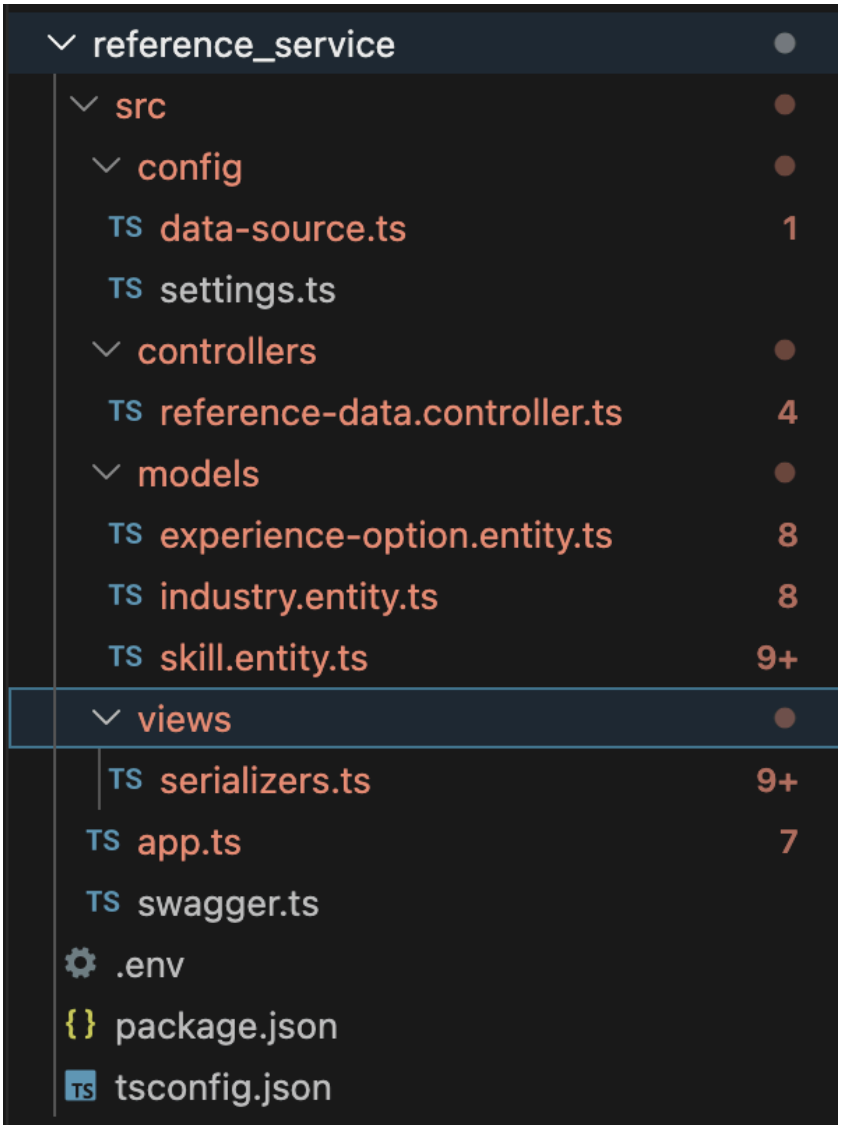
Задача

Опираясь на документ, сформированный в рамках ДЗ4, реализовать разделение бэкенд-приложения на микросервисы.

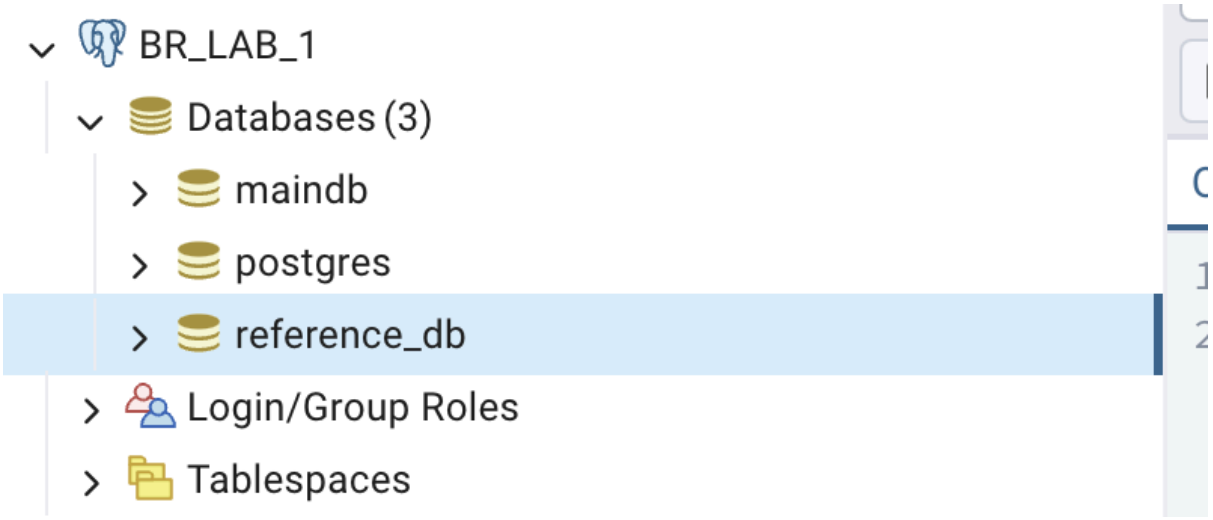
Ход работы

Reference_service

Первым делом была создана директория для 1 микросервиса Reference_service. Туда были перенесены необходимые модели и конфигурационные файлы из монолитного приложения.



Дальше, в app.ts были удалены лишние зависимости, а так же был сменён порт подключения к базе данных, а через pgAdmin была создана новая БД



После этого в моделях были убраны межсервисные связи.

reference_service > src > models > TS skill.entity.ts > Skill

```ts
import {BaseEntity, Column, Entity, OneToMany, PrimaryGeneratedColumn} from 'typeorm';

@Entity('skills')
export class Skill extends BaseEntity {
    @PrimaryGeneratedColumn({name: 'skill_id'})
    skillId: number;

    @Column({ type: 'varchar', length: 255, unique: true })
    name: string;

}
```

reference_service > src > models > TS industry.entity.ts > Industry

```ts
import {BaseEntity, Column, Entity, OneToMany, PrimaryGeneratedColumn,} from 'typeorm';

@Entity('industries')
export class Industry extends BaseEntity {
    @PrimaryGeneratedColumn({ name: 'industry_id' })
    industryId: number;

    @Column({ type: 'varchar', length: 150, unique: true })
    name: string;

}
```

reference_service > src > models > TS experience-option.entity.ts > ...

```ts
import {BaseEntity, Column, Entity, OneToMany, PrimaryGeneratedColumn, } from 'typeorm';

@Entity('experience')
export class ExperienceOption extends BaseEntity {
    @PrimaryGeneratedColumn({name: 'experience_id'})
    experienceId: number;

    @Column({ type: 'varchar', length: 100, unique: true })
    period: string;

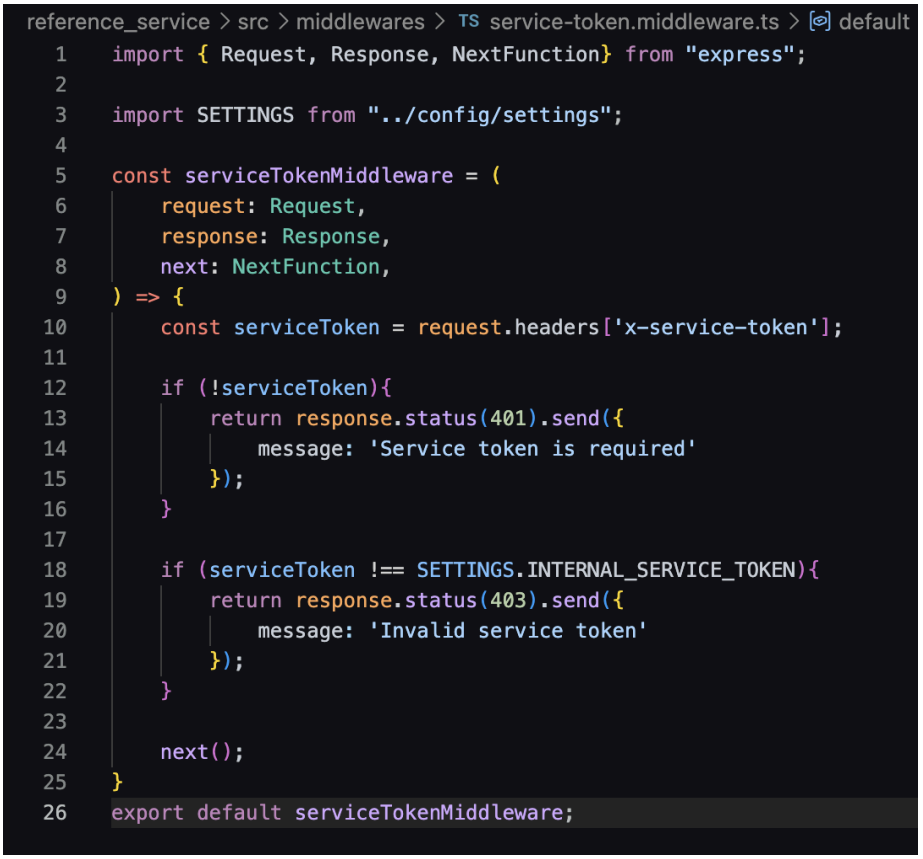
}
```

Далее были проверены и справлены контроллеры и сериализаторы. Удалены импорты из старого монолита, которые сервису уже не нужны.



После этого были установлены зависимости именно в папке микросервиса через npm install и проведён тестовый запуск.

Далее был создан middleware для проверки служебного токена, внутри происходит две проверки есть ли вообще токен при обращении и совпадает ли он.



Создаем новый котроллер, по которому другие сервисы обращаются к этому, чтобы узнать id скиллов, работ, опыта.

```
@JsonController('/internal/reference')
@UseBefore(serviceTokenMiddleware)
class InternalReferenceController{
    private experienceRepository = dataSource.getRepository(ExperienceOption);
    private industryRepository = dataSource.getRepository(Industry);
    private skillRepository = dataSource.getRepository(Skill);

    @Post('/resolve')
    async resolve (@Body() body: ResolveReferenceDto,){
        const experienceIds = body.experience_ids || [];
        const industryIds = body.industry_ids || [];
        const skillIds = body.skill_ids || [];

        const experiences = experienceIds.length > 0
            ? await this.experienceRepository.findBy({ experienceId: In(experienceIds) })
            : [];

        const industries = experienceIds.length > 0
            ? await this.industryRepository.findBy({ industryId: In(industryIds) })
            : [];

        const skills = experienceIds.length > 0
            ? await this.skillRepository.findBy({ skillId: In(skillIds) })
            : [];

        if(experiences.length !== experienceIds.length){
            throw new BadRequestError('Один или несколько experience_id не существуют');
        }

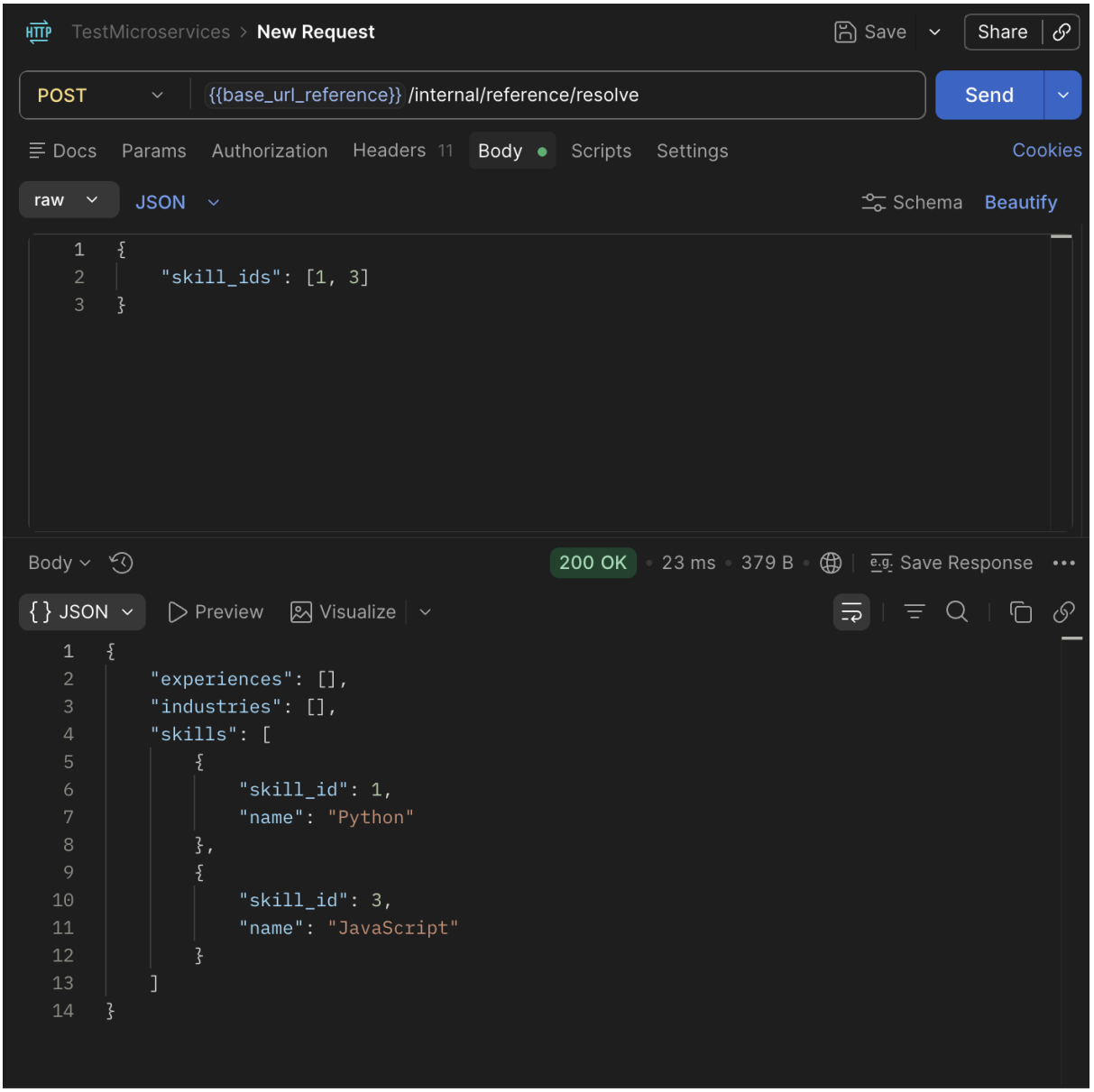
        if(industries.length !== industryIds.length){
            throw new BadRequestError('Один или несколько industry_id не существуют');
        }

        if(skills.length !== skillIds.length){
            throw new BadRequestError('Один или несколько skill_id не существуют');
        }

        return{
            experiences: experiences.map(serializeExperienceOption),
            industries: industries.map(serializeIndustry),
            skills: skills.map(serializeSkill),
        };

    }
}
export default InternalReferenceController;
```

Тесты показали, что эндпоинт работает как ожидалось и защищен служебным токеном



User_service

В папку для нового сервиса была полностью скопирована логика монолита, позже будет удаляться ненужное. Сначала поменяли название БД и порт на 8001.

В измениях моделей были сделаны следующие изменения:

Удалены связи работодателя с вакансией



Так же удалена связь резюме со скилами, так как теперь у нас таблица скилов лежит в другом микросервисе, вместо связи теперь просто колонка



Теперь в котроллере добавил ункцию обращения к микросервису скиллов, теперь мы делаем fetch по адресу

```typescript
private async resolveSkills(skillIds: number[]): Promise<ResolvedSkill[]> {
    if (skillIds.length === 0) {
        return [];
    }

    let response: Response;
    try {
        response = await fetch(`${SETTINGS.REFERENCE_SERVICE_URL}/internal/reference/resolve`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'X-Service-Token': SETTINGS.INTERNAL_SERVICE_TOKEN,
            },
            body: JSON.stringify({ skill_ids: skillIds }),
        },);
    } catch (error) {
        throw new HttpError(503, 'Reference Service недоступен');
    }

    if (response.status === 400){
        throw new NotFoundError('Один или несколько skill_id не найдены');
    }

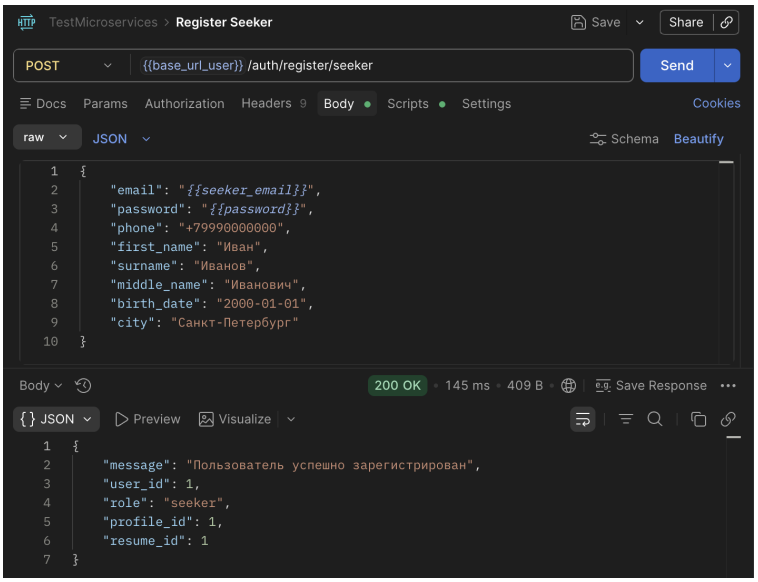
    if (!response.ok) {
        throw new HttpError(503, 'Ошибка при обращении к Reference Service');
    }
    const data = await response.json() as ReferenceResolveResponse;

    return data.skills;
}
```

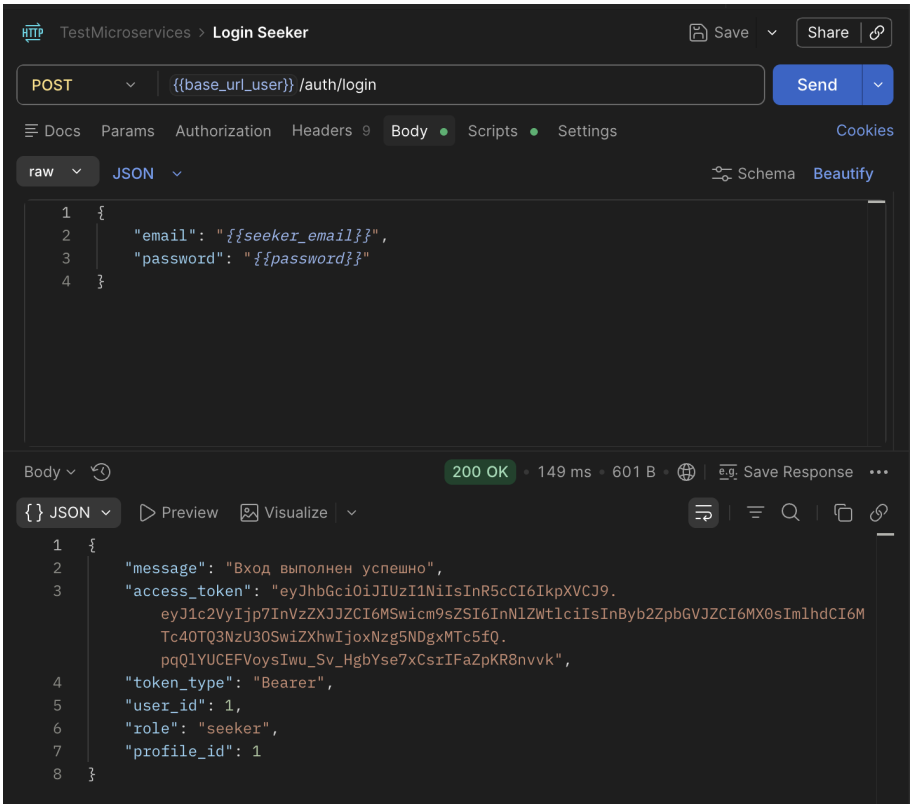
И теперь везде, где мы брали из связанной таблицы скилы, мы просто заменяем на обращение к другому микросервису.

Теперь проверяем, что микросервис работает:

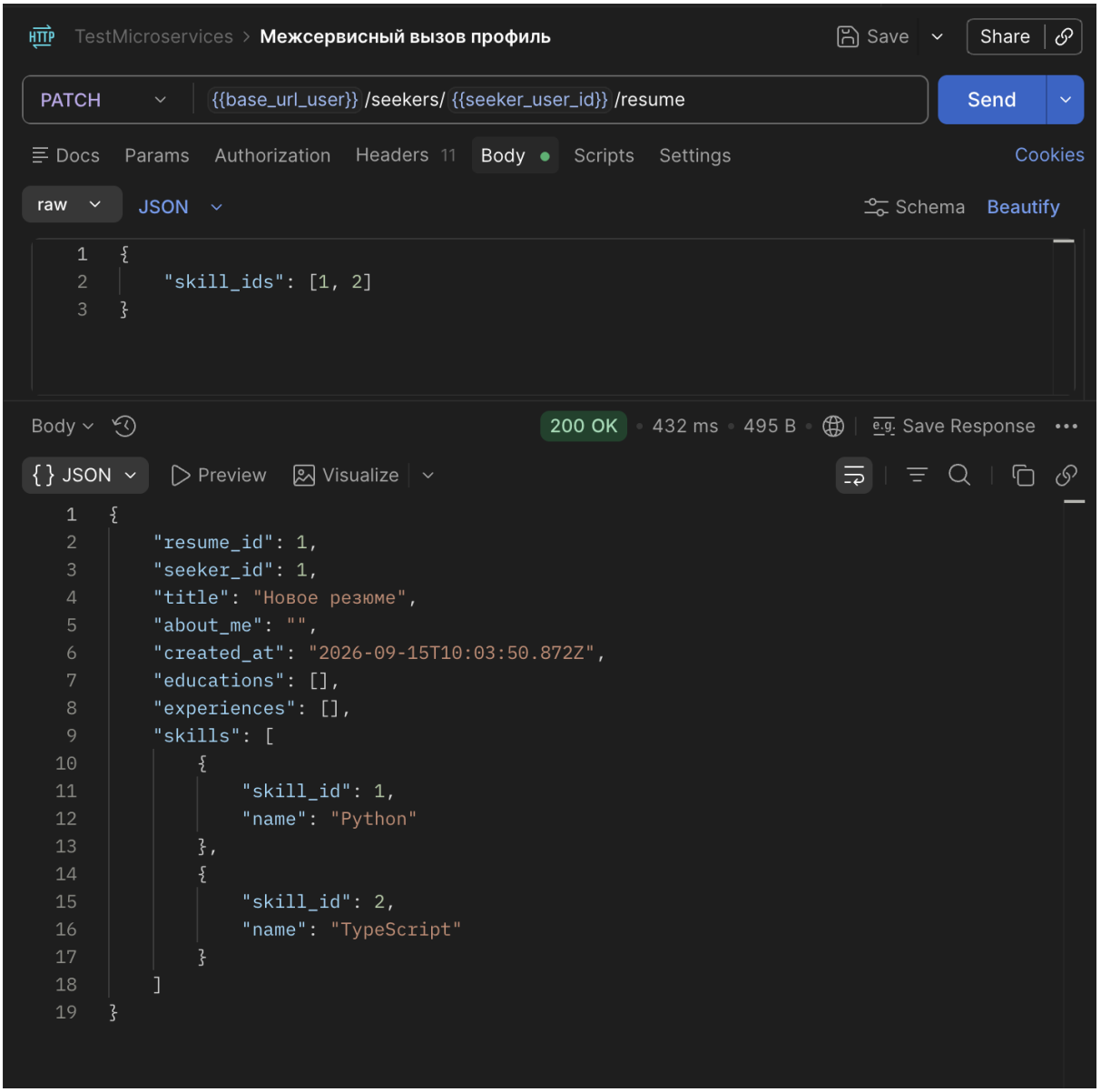
Регистрация



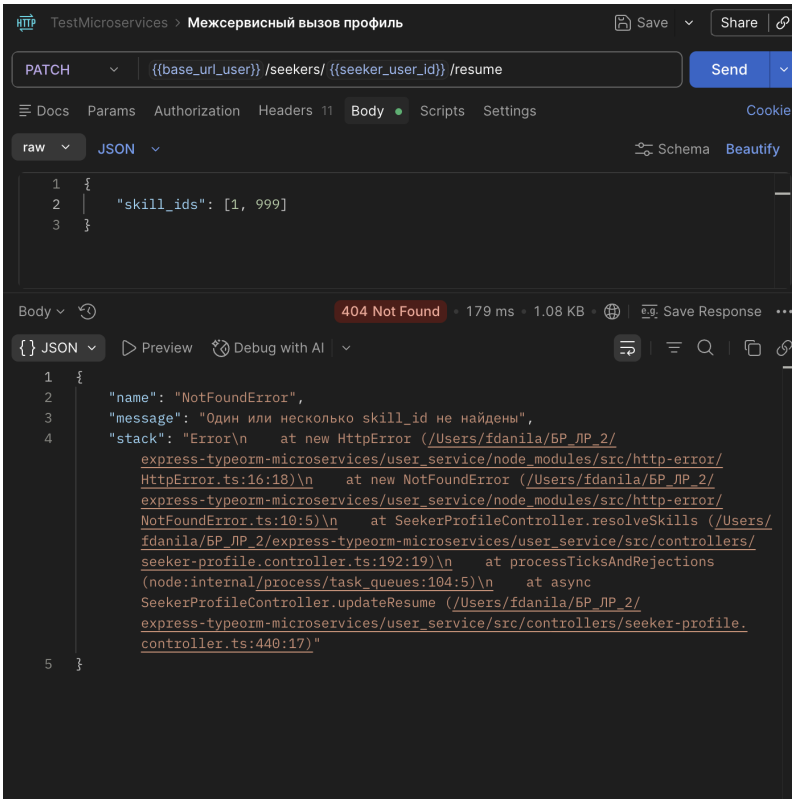
Логин



Теперь проверим межсервисный вызов



Теперь проверяем ошибку, вернулась ошибка из другуго сервиса, как и ожидалось.



Далее создадим 2 новых эндпоинта и один batchendpoin:

Первый эдпоинт: /internal/employers/:employerId

Он будет нужен для Vacancy Service для проверки того, что работодатель с каким-то id действительно существует

```
@Get('/employers/:employerId')
async getEmployer( @Param('employerId') employerId: number){
    const employer = await this.employerRepository.findOne({
        where: {
            profileId: Number(employerId)
        },
    });

    if (!employer){
        throw new NotFoundError('Работодатель не найден');
    }

    return{
        employer_id: employer.profileId,

        company_name: employer.companyName
    };
```

Второй эндпоинт /internal/resumes/:resumeId

Он нужен для Response Service, когда соискатель попытается откликнуться на вакансию, то Response Service сможет получить резюме этого соискателя. Также сможет сверить id из токена и id того, кто сейчас создает отклик.

```
@Get('/resumes/:resumeId')
async getResume(@Param('resumeId') resumeId: number){
    const resume = await this.resumeRepository.findOne({
        where: {
            resumeId: Number(resumeId)
        },
        relations: {
            seeker: {
                user: true,
            },
        },
    });

    if(!resume) {
        throw new NotFoundError('Резюме не найдено');
    }
    return{
        resume_id: resume.resumeId,
        seeker_id: resume.seeker.profileId,
        title: resume.title,
        seeker: {
            first_name: resume.seeker.firstName,
            surname: resume.seeker.surname,
            middle_name: resume.seeker.middleName,
            email: resume.seeker.user.email,
            phone: resume.seeker.user.phone,
        }
    }

}
```

Третий batch эндпоинт POST internal/resumes/resolve

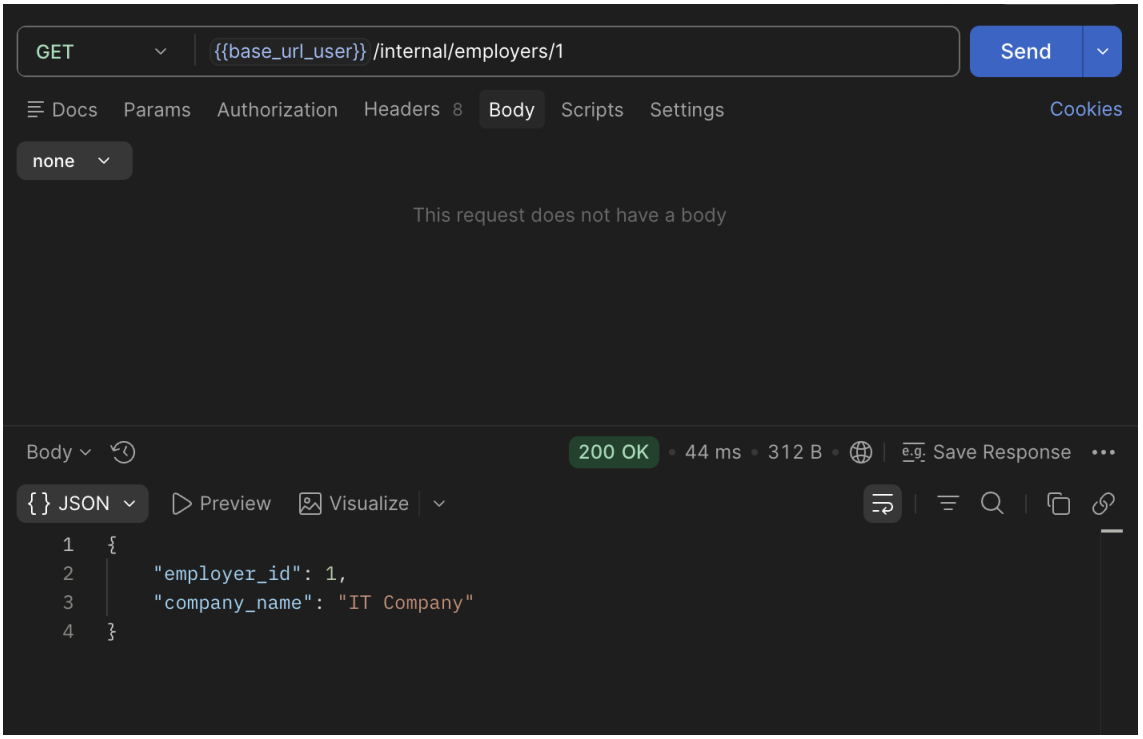
Он будет нужен, когда работодатель захочет посмотреть список откликов, чтобы не делать кучу запросов, нужен этот эндпоинт, чтобы подтянуть сразу все необходимые отклики.

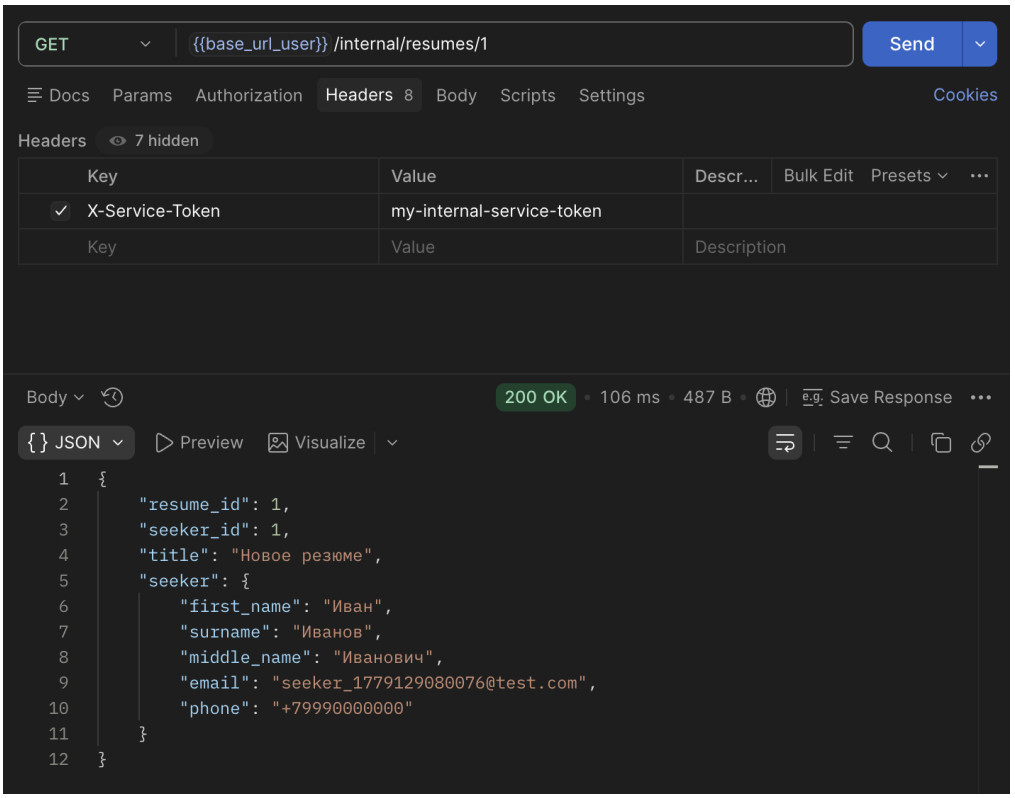
```
@Post('/resume/resolve')
async resolveResumes(@Body() body: ResolveResumeDto){
    if (body.resume_ids.length === 0){
        return{items: []};
    }
    const resumes = await this.resumeRepository.find({
        where: {
            resumeId: In(body.resume_ids)
        },
        relations: {
            seeker: true,
        },
    });

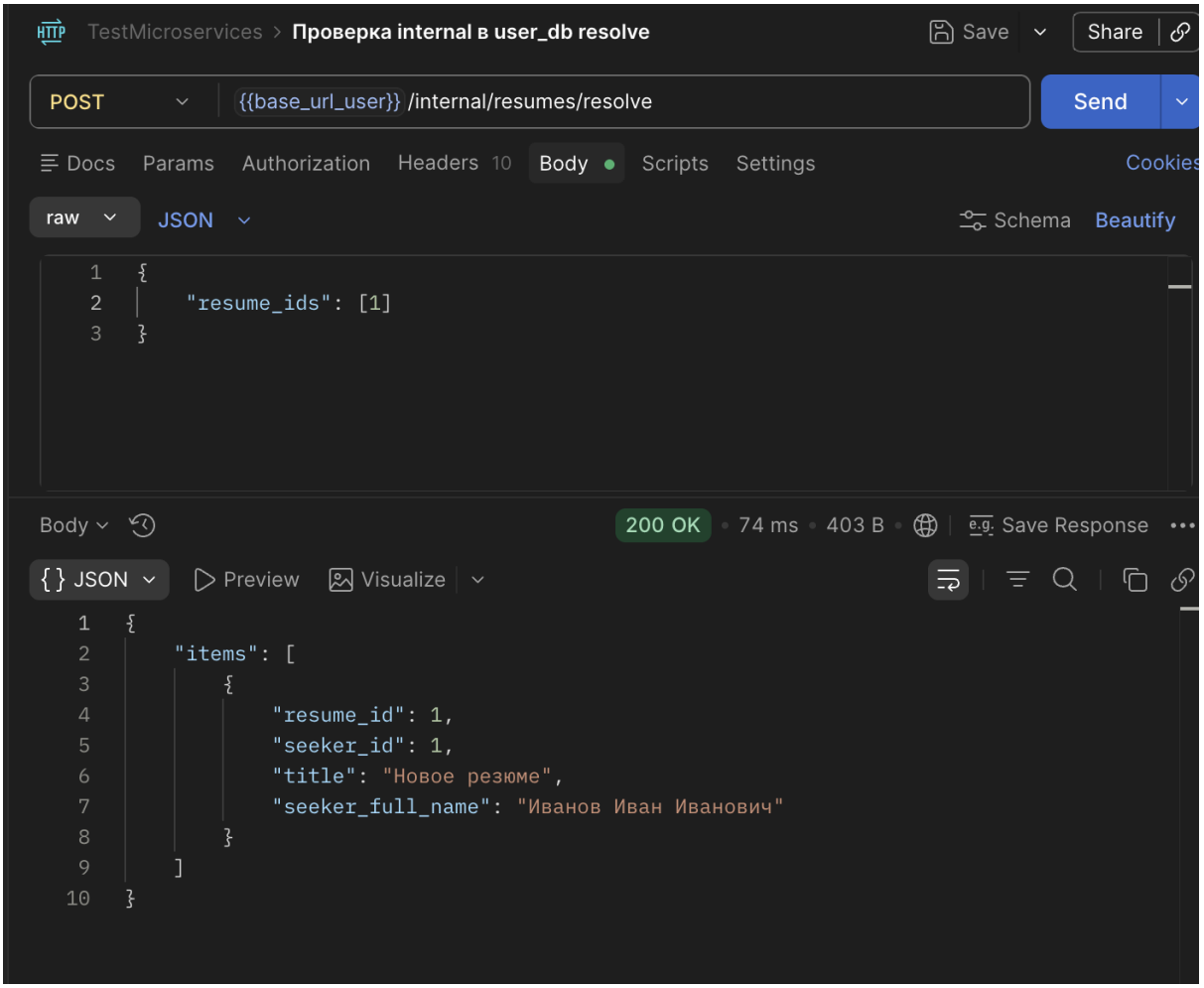
    if (resumes.length !== body.resume_ids.length){
        throw new BadRequestError('Одно или несколько резюме не найдены');
    }

    return{items: resumes.map(
     (resume) => ({
        resume_id: resume.resumeId,
        seeker_id: resume.seeker.profileId,
        title: resume.title,
        seeker_full_name:[
            resume.seeker.surname,
            resume.seeker.firstName,
            resume.seeker.middleName,
        ].filter(Boolean).join(' '),
     }),
    )};
}
```

Далее тесты:


GET
{{base_url_user}} /internal/employers/1
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
none
This request does not have a body
Body
200 OK
44 ms
312 B
Save Response
JSON
Preview
Visualize
{
"employer_id": 1,
"company_name": "IT Company"
}

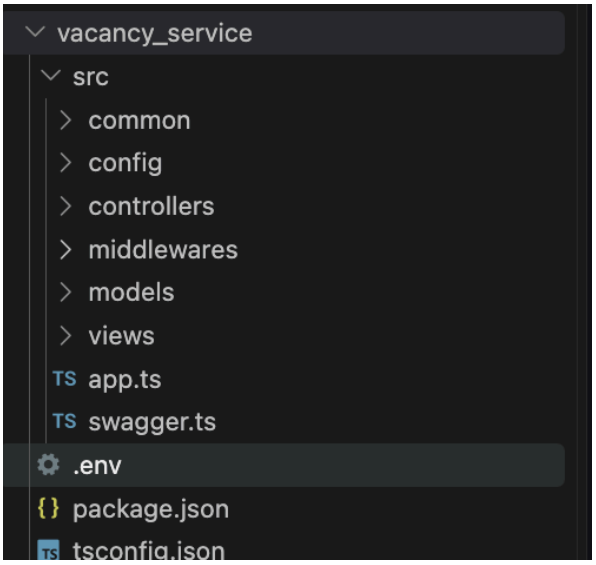

GET
{{base_url_user}} /internal/resumes/1
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Headers
7 hidden
Key
Value
Descr...
Bulk Edit
Presets
X-Service-Token
my-internal-service-token
Key
Value
Description
Body
200 OK
106 ms
487 B
Save Response
JSON
Preview
Visualize
{
"resume_id": 1,
"seeker_id": 1,
"title": "Новое резюме",
"seeker": {
"first_name": "Иван",
"surname": "Иванов",
"middle_name": "Иванович",
"email": "seeker_1779129080076@test.com",
"phone": "+79990000000"
}
}



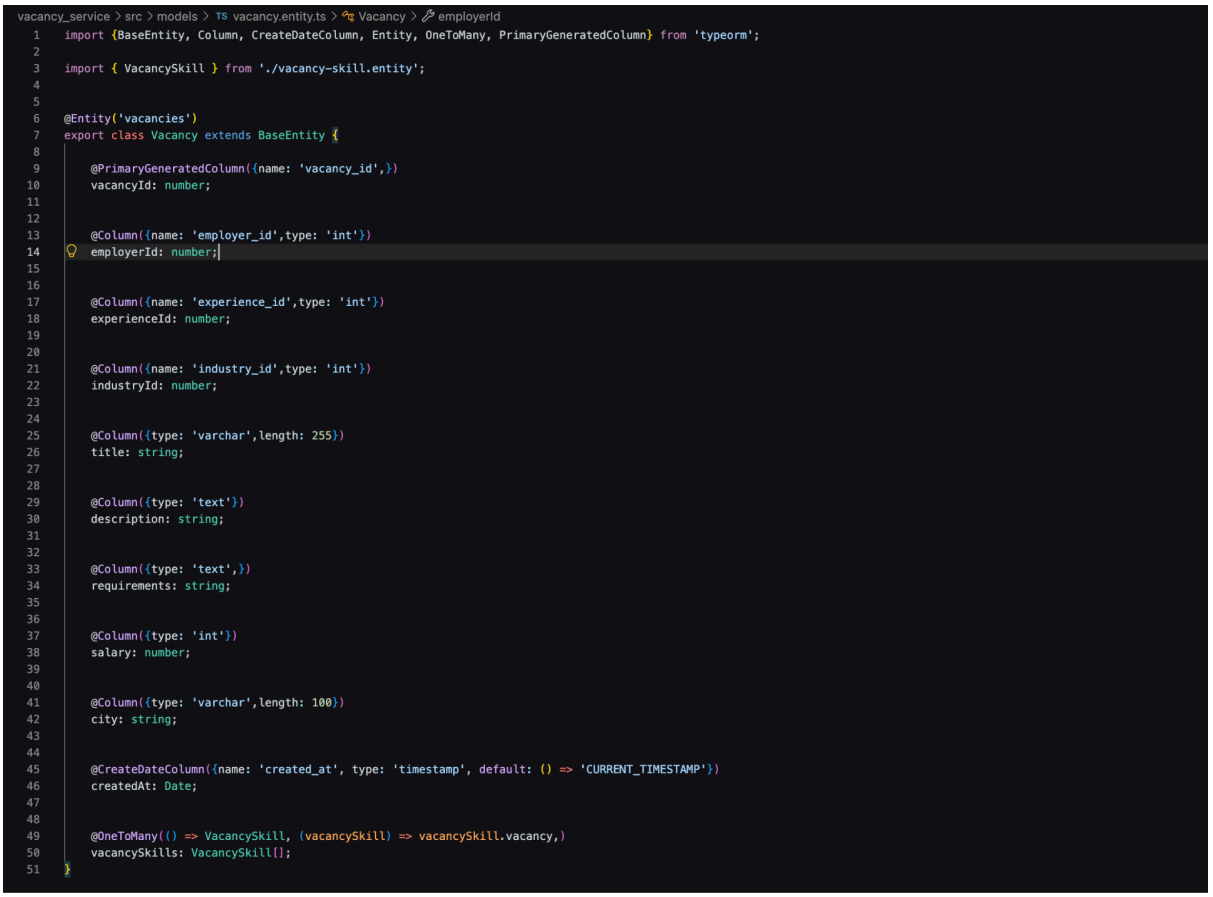
Все эндпоинты прошли проверку

Vacancy Service

Создаем папку для нового микросервиса по аналогии с предыдущими



Исправим с моделях поля, которые остались от предыдущей БД



vacancy_service > src > models > TS vacancy-skill.entity.ts > ...

```ts
import {BaseEntity, Entity, JoinColumn, ManyToOne, PrimaryGeneratedColumn, Unique, Column} from 'typeorm';
import { Vacancy } from './vacancy.entity';

@Entity('vacancy_skills')
@Unique('uq_vacancy_skill', ['vacancy', 'skillId'])
export class VacancySkill extends BaseEntity {
    @PrimaryGeneratedColumn({ name: 'id' })
    id: number;

    @ManyToOne(() => Vacancy, (vacancy) => vacancy.vacancySkills, {onDelete: 'CASCADE'})
    @JoinColumn({ name: 'vacancy_id' })
    vacancy: Vacancy;

    @Column({ name: 'skill_id', type: 'int' })
    skillId: number;
}
```

Теперь добавляем в старый контроллер из монолита http helper lля обращения к User Service

```ts
//http хэлпер
private async getEmployer(employerId:number): Promise<ResolvedEmployer> {
    let response: Response;
    try {
        response = await fetch(`${SETTINGS.USER_SERVICE_URL}/internal/employers/${employerId}`,
            {
                headers: {
                    'X-Service-Token': SETTINGS.INTERNAL_SERVICE_TOKEN,
                },
            },
        );
    } catch (error) {throw new HttpError(503, 'User Service недоступен');}

    if (response.status === 404) {
        throw new NotFoundError('Работодатель не найден');
    }

    if (!response.ok) {
        throw new HttpError(503, 'Ошибка взаимодействия с User Service');
    }
    return await response.json() as ResolvedEmployer;
}
```

Такой же делаем для Reference Service

```ts
//http хэлпер для Reference Service
private async resolveReferenceData(experienceIds: number[] = [], industryIds: number[] = [], skillIds: number[] = []): Promise<ReferenceResolveResponse> {
    if (experienceIds.length === 0 && industryIds.length === 0 && skillIds.length === 0) {
        return {
            experiences: [],
            industries: [],
            skills: [],
        };
    }

    let response: Response;
    try {
        response = await fetch(`${SETTINGS.REFERENCE_SERVICE_URL}/internal/reference/resolve`,{
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'X-Service-Token': SETTINGS.INTERNAL_SERVICE_TOKEN,
            },
            body: JSON.stringify({
                experience_ids: experienceIds,
                industry_ids: industryIds,
                skill_ids: skillIds,
            }),
        });
    } catch (error) {
        throw new HttpError(503, 'Reference Service недоступен');
    }

    if (response.status === 404) {
        throw new NotFoundError('Один или несколько элементов справочника не найдены');
    }

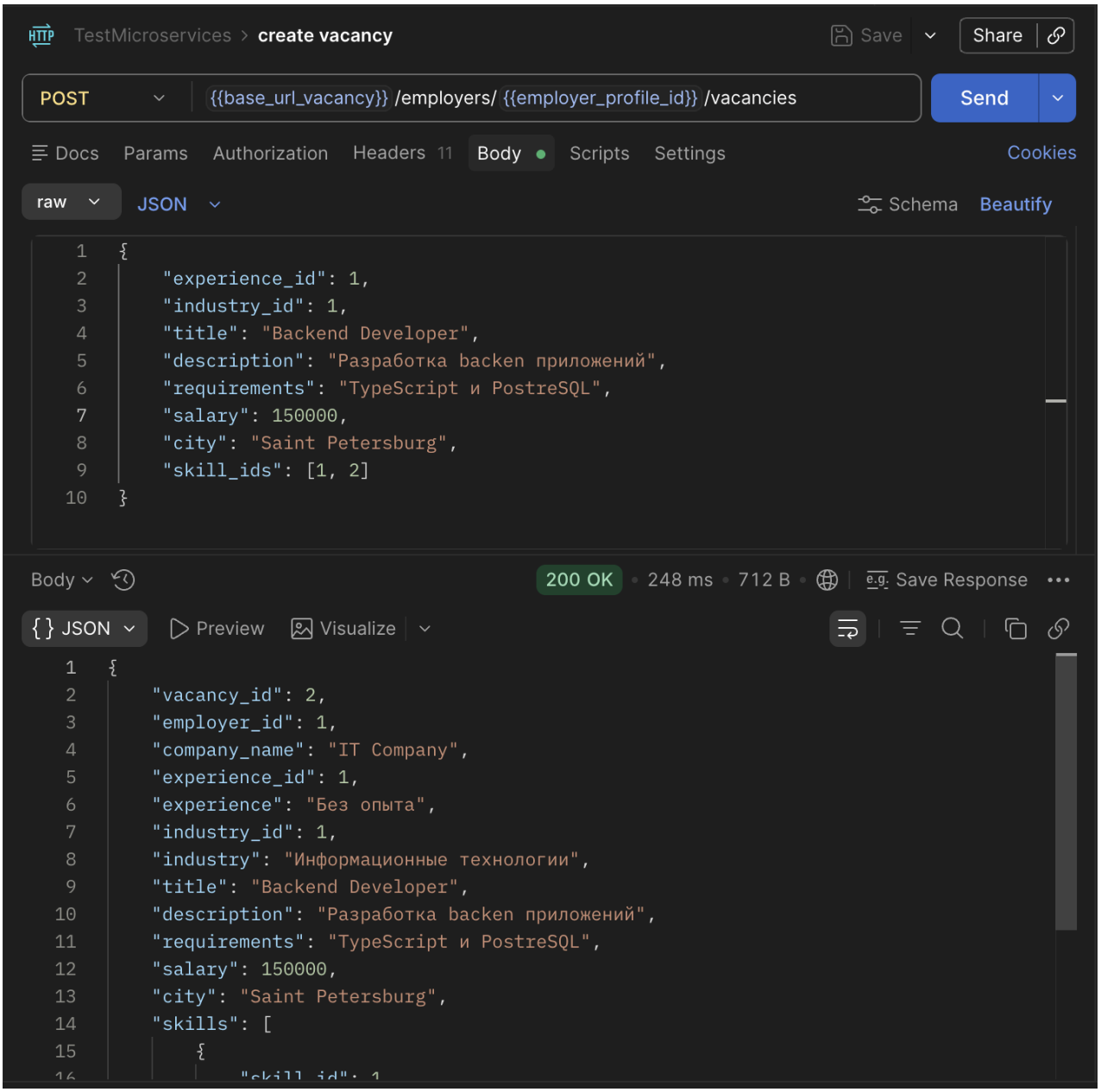
    if (!response.ok) {
        throw new HttpError(503, 'Ошибка взаимодействия с Reference Service');
    }

    return await response.json() as ReferenceResolveResponse;
}
```

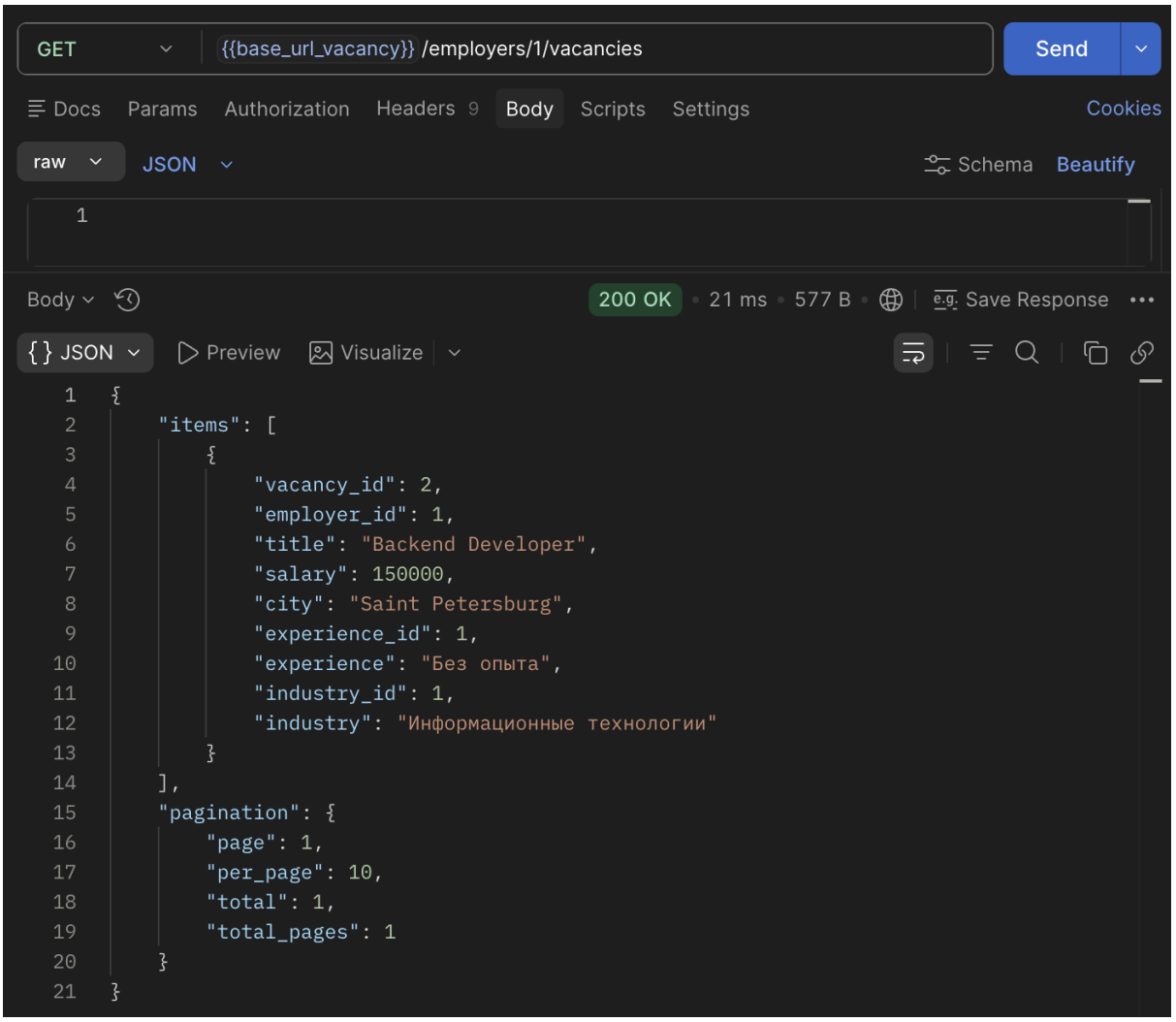
После этого из контроллера и его эндпоинтов были удалены связи с моделями, которых больше нет. Теперь обращение происходит через http к другим микросервисам.

Перейдем к проверкам:

Создание вакансии работает



Получение конкретной вакансии тоже работает, при этом связанные даные получаются от другого микросервиса



Так как публичная часть эндпоинтов прошла проверку, то перейдем к внутренним эндпоинтам.

Первый внутренний контроллер /internal/vacancies/:vacancyId нужен, чтобы получать информацию о вакансии по id

/internal/vacancies/resolve нужен для того случая, когда у работодателя есть десятки откликов на вакансии, чтобы можно было найти информацию об каждой вакансии и не делать множество запросов и нужен этот эндпоинт

```
@JsonController('/internal/vacancies')
@UseBefore(serviceTokenMiddleware)
class InternalVacancyController {
    private vacancyRepository = dataSource.getRepository(Vacancy);


    @Get('/:vacancyId')
    async getVacancy(@Param('vacancyId') vacancyId: number){
        const vacancy = await this.vacancyRepository.findOne({
            where: {
                vacancyId: Number(vacancyId),
            },
        });

        if(!vacancy){throw new NotFoundError('Вакансия не найдена')}

        return{
            vacancyId: vacancy.vacancyId,
            employerId: vacancy.employerId,
            title: vacancy.title

        };


    }
```

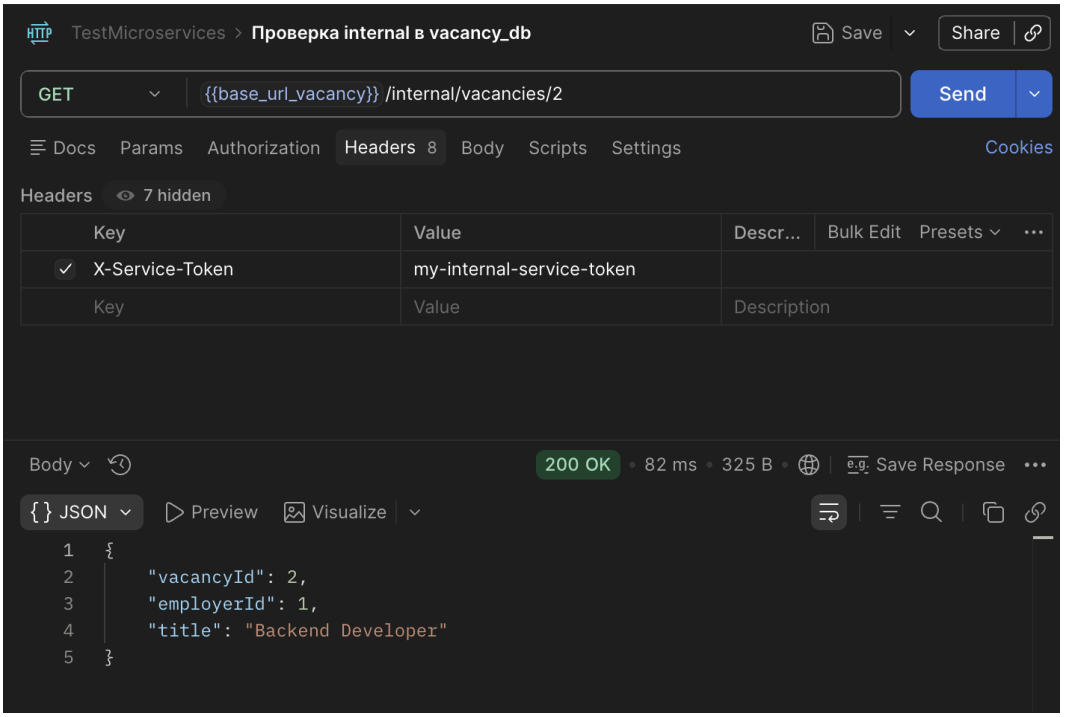
```
  @Post('/resolve')
  async resolveVacancies(@Body() body: ResolveVacanciesDto){
      if (body.vacancy_ids.length === 0){
          return {items: []}
      }

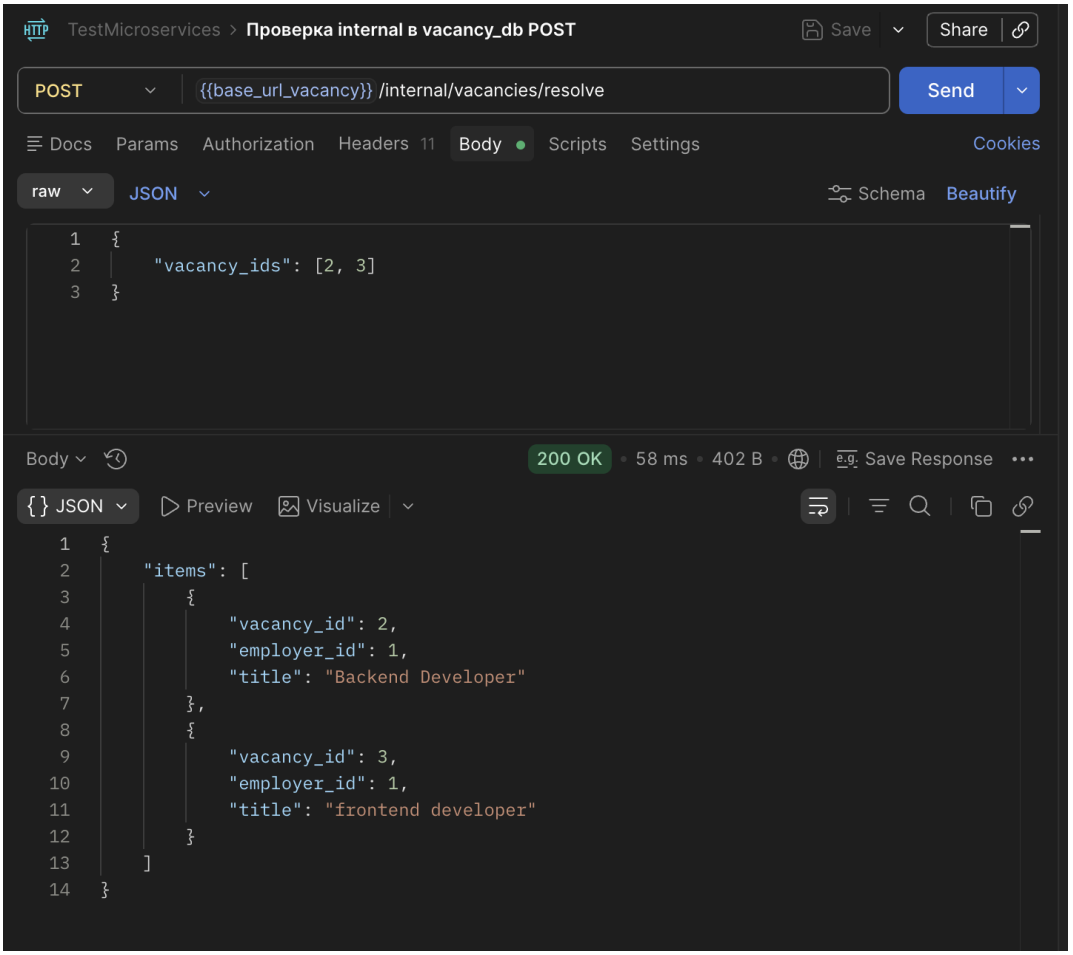
      const vacancies = await this.vacancyRepository.find({
          where: {
              vacancyId: In(body.vacancy_ids),
          },
      });

      if(vacancies.length !== body.vacancy_ids.length){
          throw new BadRequestError('Одна или несколько вакансий не найдены')
      }

      return{
          items: vacancies.map((vacancy) => ({
              vacancy_id: vacancy.vacancyId,
              employer_id: vacancy.employerId,
              title: vacancy.title
          }))
      }
  }
```

Переходим к тестам:





Response Service

Как обычно пересобираем модель, чтобы удалить связи с БД, которых нет в микросервисе, заменяем необходимые поля на id.

response_service > src > models > TS response.entity.ts > ApplicationResponse

```ts
import {BaseEntity, Column, CreateDateColumn, Entity, JoinColumn, ManyToOne, OneToOne, PrimaryGeneratedColumn, Unique} from 'typeorm';
import { ResponseStatus } from './enums';
import { CoverLetter } from './cover-letter.entity';

@Entity('responses')
@Unique('uq_vacancy_resume_response', ['vacancyId', 'resumeId'])
export class ApplicationResponse extends BaseEntity {
    @PrimaryGeneratedColumn({ name: 'response_id' })
    responseId: number;

    @Column({name: 'vacancy_id', type: 'int'})
    vacancyId: number;


    @Column({name: 'resume_id', type: 'int'})
    resumeId: number;


    @Column({name: 'seeker_id', type: 'int'})
    seekerId: number;


    @Column({name: 'employer_id', type: 'int'})
    employerId: number;


    @Column({type: 'enum', enum: ResponseStatus, default: ResponseStatus.NEW})
    status: ResponseStatus;

    @CreateDateColumn({ name: 'created_at', type: 'timestamp', default: () => 'CURRENT_TIMESTAMP' })
    createdAt: Date;

    @OneToOne(() => CoverLetter, (coverLetter) => coverLetter.response)
    coverLetter?: CoverLetter;
}
```

Создаем хелперы, которые позволят пользоваться служебными эндпоинтами, первый ищет вакансию по её id.

```ts
//хелпер для обращения к vacancy service
private async getVacancy(vacancyId: number): Promise<ResolvedVacancy>{
    let response: Response;

    try {
        response = await fetch(`${SETTINGS.VACANCY_SERVICE_URL}/internal/vacancies/${vacancyId}`,
            {
                headers: {
                    'X-Service-Token': SETTINGS.INTERNAL_SERVICE_TOKEN,
                },
            },
        );
    } catch {throw new HttpError(503, 'Vacancy Service недоступен');
    }

    if (response.status === 404) {
        throw new NotFoundError(
            'Вакансия не найдена',
        );
    }

    if (!response.ok) {
        throw new HttpError(503, 'Ошибка Vacancy Service');
    }

    return await response.json() as ResolvedVacancy;
}
```

Второй хелпер позволяет найти резюме по его id.

```
//хелпер для обращения к user service

private async getResume(resumeId: number): Promise<ResolvedResume> {
    let response: Response;

    try {
        response = await fetch(`${SETTINGS.USER_SERVICE_URL}/internal/resumes/${resumeId}`,
            {
                headers: {
                    'X-Service-Token': SETTINGS.INTERNAL_SERVICE_TOKEN,
                },
            },
        );
    } catch {throw new HttpError(503, 'User Service недоступен')}

    if (response.status === 404) {
        throw new NotFoundError(
            'Резюме не найдено',
        );
    }

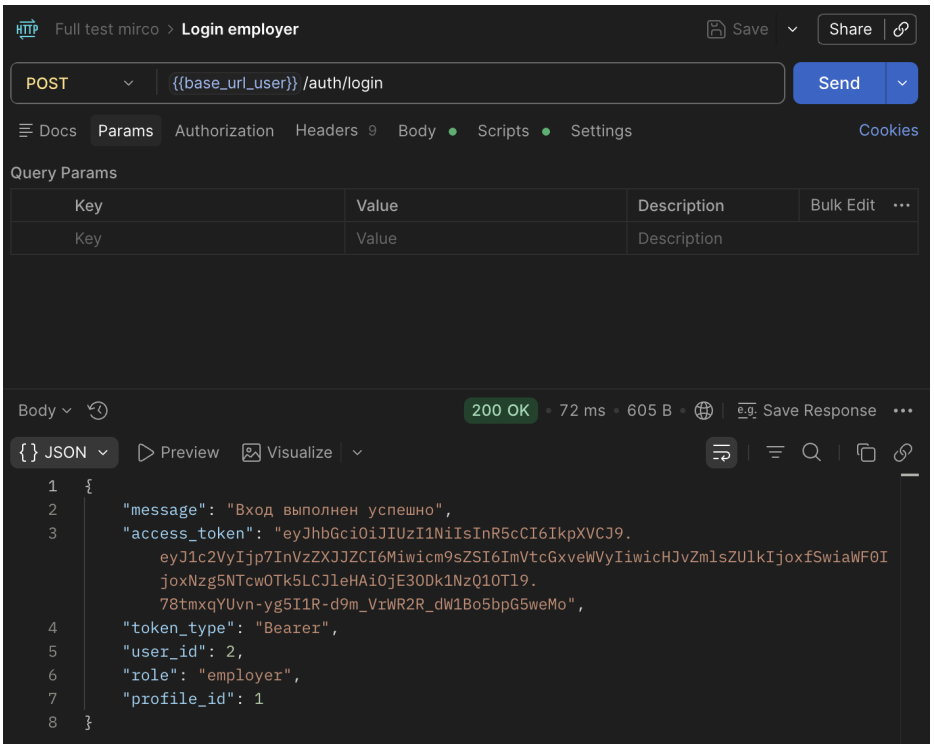
    if (!response.ok) {
        throw new HttpError(503, 'Ошибка User Service');
    }

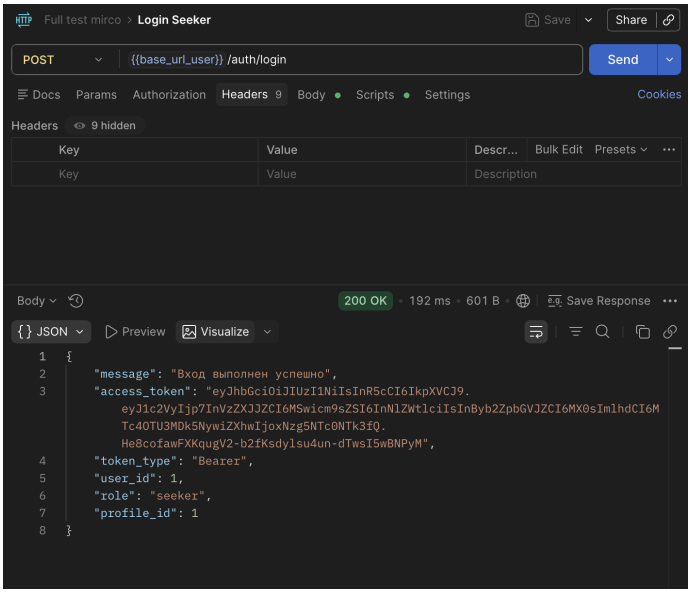
    return await response.json() as ResolvedResume;
}
```

Далее во всех публичных эндпоинтах заменяем обращение к связанным таблицам на наши хелперы.

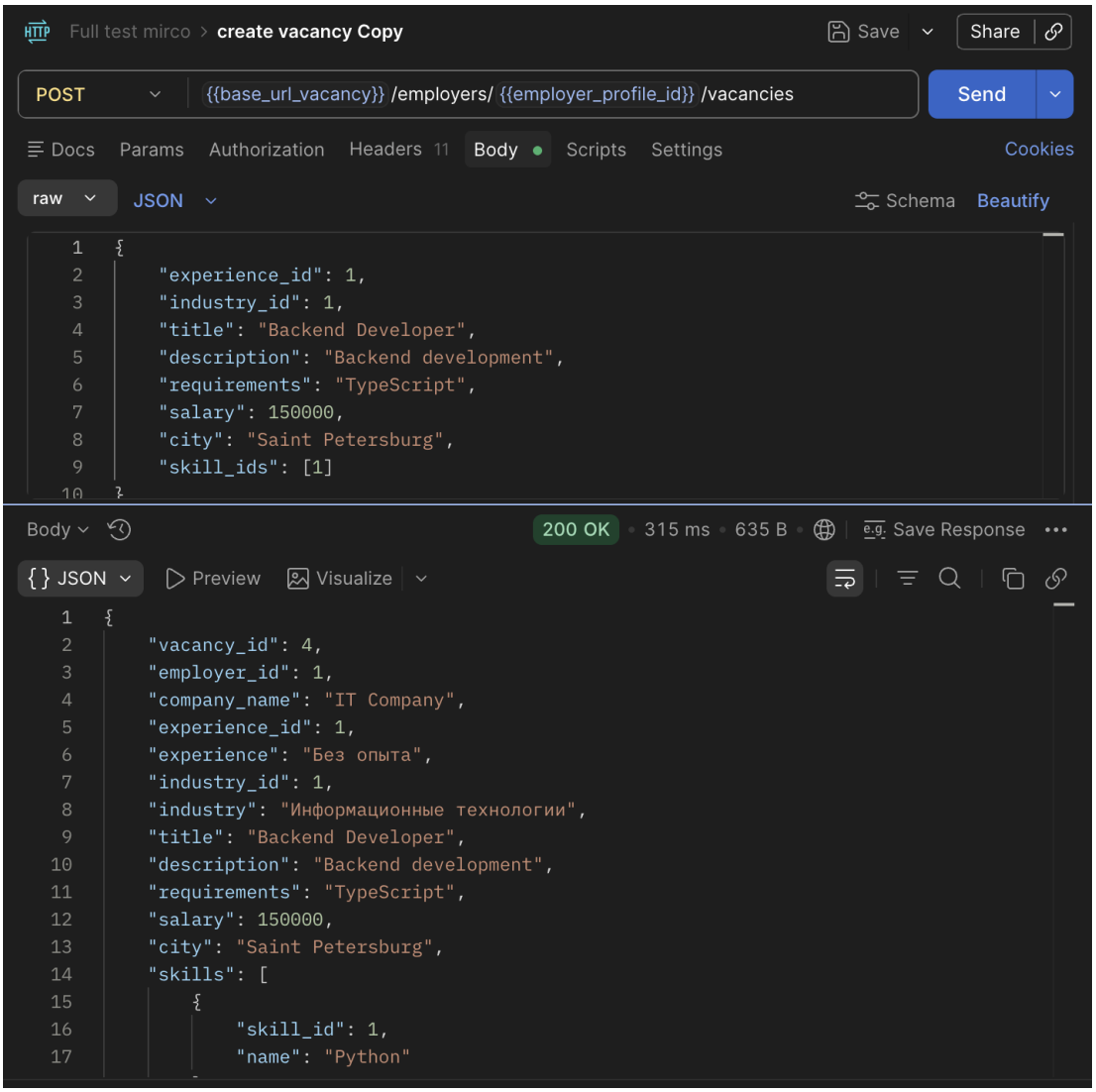
Тестирование системы

Выполняем логины от работодателя и соискателя

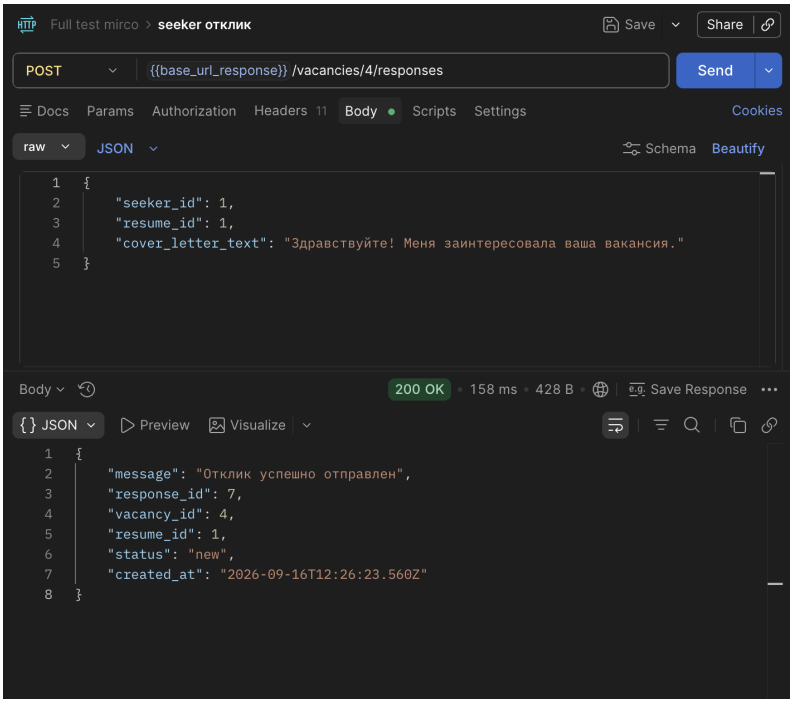




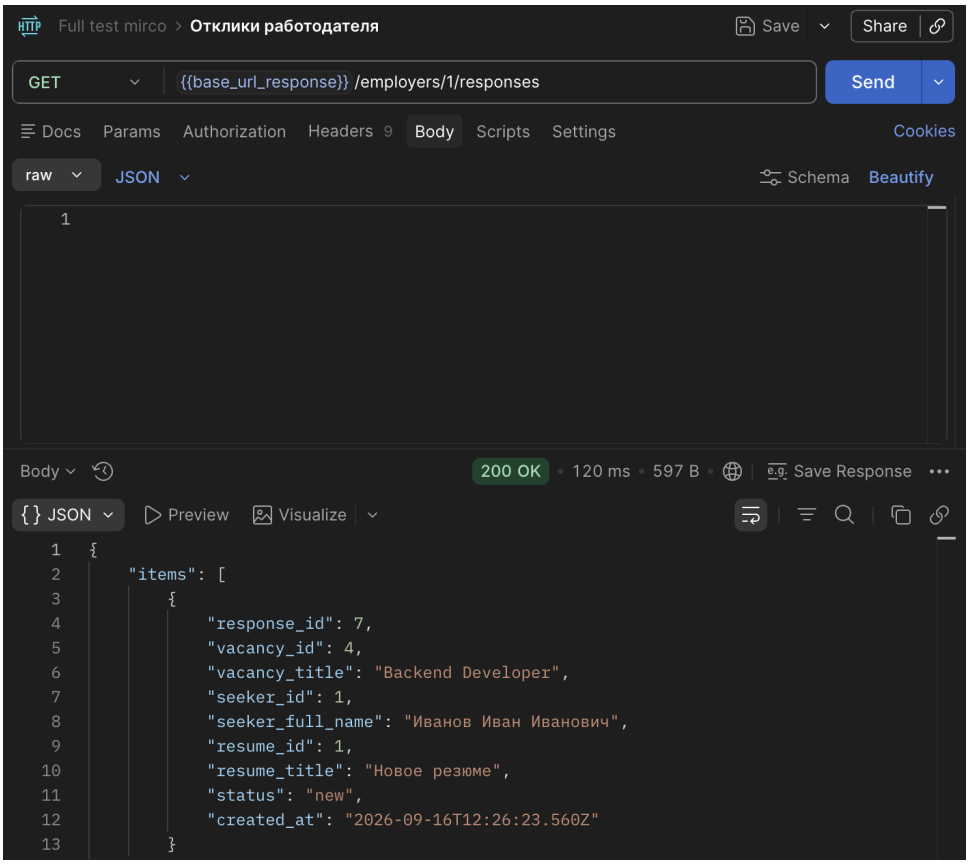
Далее employer создает вакансию



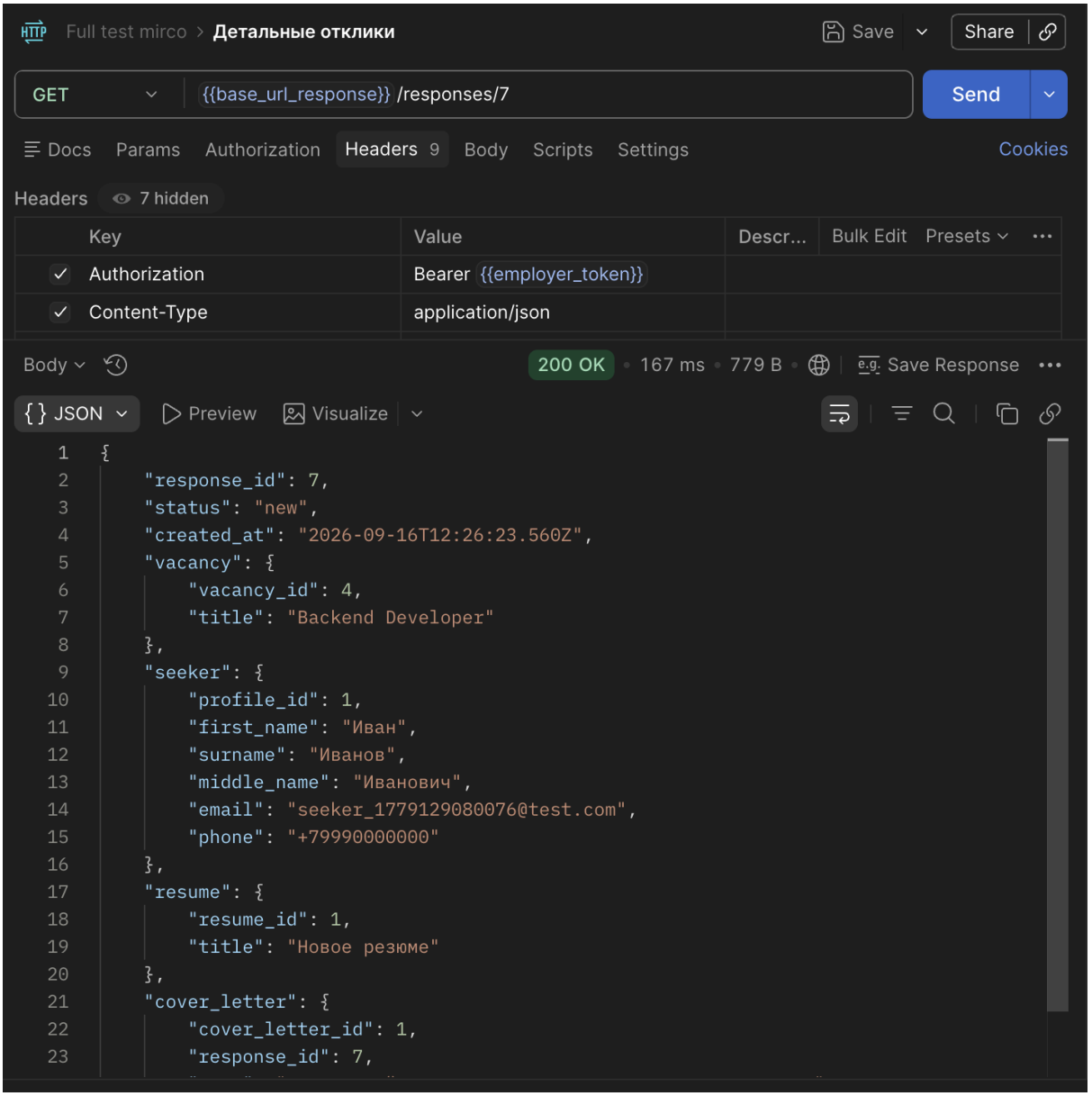
Далее Seeker откликается на вакансию



Работодатель смотрит отклики



Также работодатель смотрит детальный отклик



Вывод

В ходе лабораторной работы монолитное приложение было разделено на четыре микросервиса: User Service, Reference Service, Vacancy Service и Response Service. Для каждого сервиса была выделена отдельная база данных, а межсервисные связи через внешние ключи были заменены хранением идентификаторов и взаимодействием через внутренний REST API. Также была реализована проверка межсервисных запросов с помощью X-Service-Token. В результате была получена работоспособная микросервисная архитектура, в которой каждый сервис отвечает только за свою область данных и может взаимодействовать с другими сервисами независимо.